

ASSIGNMENT-8.2

NAME: ARCHITHA

ROLLNO: 2303A51001

BATCH:30

TASK-1

PROMPT:

Write a function `is_even()` generate test cases for finding if a number is even or odd

CODE:

The image shows a VS Code editor window with the following components:

- Explorer:** A sidebar on the left showing the file structure. The file `Assignment-8.2.py` is selected under the `AI ASSIGNMENTS` folder.
- Editor:** The main workspace displays the content of `Assignment-8.2.py`. The code defines a function `is_even` and includes test cases.
- Terminal:** A terminal window at the bottom shows the command `python Assignment-8.2.py` being executed, resulting in the output: `True, False, True, True, False`.

```
1 #Write a function is_even() generate test cases for finding if a number is even or odd.
2 def is_even(num):
3     "return True if the number is even, False otherwise"
4     return num % 2 == 0
5
6 #test cases
6 print(is_even(2)) # True
7 print(is_even(7)) # False
8 print(is_even(0)) # True
9 print(is_even(-4)) # True
10 print(is_even(1)) # False
11
```

Terminal Output:

```
PS E:\User\Jashwanth\Desktop\AI ASSIGNMENTS> & C:\Users\Jashwanth\AppData\Local\Microsoft\WindowsApps\python3.12.exe "e:/User/jashwanth/desktop/AI ASSIGNMENTS/Assignment-8.2.py"
True
False
True
True
False
PS E:\User\Jashwanth\Desktop\AI ASSIGNMENTS>
```

```

1 # Write a function is_even() generate test cases for finding if a number is even or odd.
2 import unittest
3
4
5 def is_even(num):
6     "Return True if the number is even, False otherwise"
7     return num % 2 == 0
8
9 # Test cases
10 print(is_even(2)) # True
11 print(is_even(7)) # False
12 print(is_even(0)) # True
13 print(is_even(-4)) # True
14 print(is_even(9)) # False
15 # ASSERT TEST CASES
16 assert is_even(2) == True
17 assert is_even(3) == False
18 assert is_even(0) == True
19 print("All assert test cases passed")
20 # EXCEPTION TEST CASE
21 try:
22     is_even("a")
23 except TypeError:
24     print("Exception test passed")
25 # UNIT TEST CASES
26 class TestIsEven(unittest.TestCase):
27     def test_even(self):
28         self.assertEqual(is_even(2), True)
29         self.assertEqual(is_even(0), True)
30         self.assertEqual(is_even(-4), True)
31     def test_odd(self):
32         self.assertEqual(is_even(1), False)
33         self.assertEqual(is_even(11), False)
34 # PYTEST STYLE TEST CASES
35 def test_even_numbers():
36     assert is_even(2) == True
37     assert is_even(0) == True
38 def test_odd_numbers():
39     assert is_even(2) == False
40     assert is_even(11) == False
41 if __name__ == "__main__":
42     unittest.main()
43 def test_odd_numbers():
44     assert is_even(3) == False
45     assert is_even(11) == False
46 # DOC TEST
47 def is_even(num):
48     ---

```

PYTEST:

```

File "C:\Program Files\WindowsApps\PythonSoftwareFoundation.Python.3.12_3.12.2000.0_x64_gz5n2kfralpg@l1b\doctest.py", line 2845, in <module>
  sys.exit(test())
File "C:\Program Files\WindowsApps\PythonSoftwareFoundation.Python.3.12_3.12.2000.0_x64_gz5n2kfralpg@l1b\doctest.py", line 2833, in test
  m = _import._filenames[::3])
ModuleNotFoundError: No module named 'Assignment'
PS E:\User\Jashwanth\Desktop\VAI ASSIGNMENTS> python -m pytest -v Assignment-8.2.py
platform win32 -- Python 3.12.10, pytest-9.0.2, pluggy-1.6.0 -- C:\Users\Jashwanth\AppData\Local\Microsoft\WindowsApps\PythonSoftwareFoundation.Python.3.12_gz5n2kfralpg@python.exe
cachedir: .pytest_cache
rootdir: E:\User\Jashwanth\Desktop\VAI ASSIGNMENTS
collected 0 items / 1 error

===== ERRORS =====
ImportError while importing test module 'E:\User\Jashwanth\Desktop\VAI ASSIGNMENTS\Assignment-8.2.py'.

```

OBSERVATION:

1. Normal Test Case:

First, I tested the function by printing the results of `is_even()`.

Then I manually checked whether the output was correct – that is, whether the numbers were even or odd.

2. Assert Test Case:

Next, I used assert statements to automatically verify the results.

If the function returns **True for even numbers** and **False for odd numbers**, the test passes.

3. **Exception Test Case:**

I also tested the function with invalid input, like a string.

This was to make sure the function properly raises an error when the input is not an integer.

4. **unittest Test Case:**

After that, I created a unittest class.

This helped me test multiple even and odd numbers in a structured and organized way.

5. **pytest Test Case:**

I then used pytest, where I wrote simple assert statements in separate test functions to check different inputs.

6. **Doctest:**

Finally, I added example test cases inside the function's docstring.

Using doctest, these examples were automatically executed to verify that the function works correctly.

Types of test cases:

1. **Assert test case-** An assert test case uses the assert keyword to automatically verify expected output.

2. **normal test case-** A normal test case checks output using print() and requires manual verification.

3. **Exception Test Case:** An exception test case checks whether a function correctly raises an error for invalid input.

4. **unit test Test Case:** A unittest test case uses Python's unittest module to test functions inside a class structure.

5. **pytest Test Case:** A pytest test case uses the pytest framework and relies on simple assert statements.

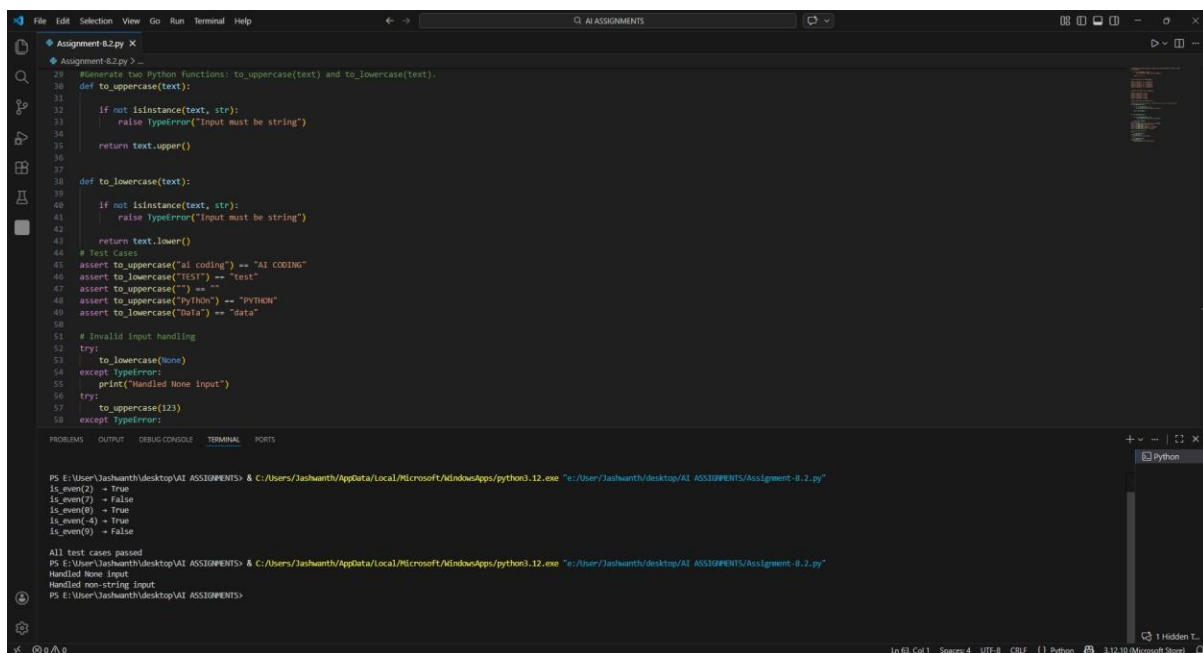
6. **Doctest:** A doctest is written inside a function's docstring and looks like an interactive Python session.

TASK-2

PROMPT:

Generate two Python functions: to_uppercase(text) and to_lowercase(text).

CODE:



```
Assignment-8.2.py X
# Generate two Python functions: to_uppercase(text) and to_lowercase(text).
def to_uppercase(text):
    if not isinstance(text, str):
        raise TypeError("Input must be string")
    return text.upper()

def to_lowercase(text):
    if not isinstance(text, str):
        raise TypeError("Input must be string")
    return text.lower()

# Test cases
assert to_uppercase("ai coding") == "AI CODING"
assert to_lowercase("TEST") == "test"
assert to_uppercase("") == ""
assert to_uppercase("python") == "PYTHON"
assert to_lowercase("Data") == "data"

# Invalid input handling
try:
    to_lowercase(None)
except TypeError:
    print("Handled None input")
try:
    to_uppercase(123)
except TypeError:
```

```
PS E:\Users\Jashwanth\Desktop\VAI ASSIGNMENTS> & C:\Users\Jashwanth\AppData\Local\Microsoft\WindowsApps\python3.12.exe "e:/User/Jashwanth/desktop/AI ASSIGNMENTS/Assignment-8.2.py"
is_even(2) -> True
is_even(7) -> False
is_even(9) -> True
is_even(4) -> True
is_even(9) -> False

All test cases passed
PS E:\Users\Jashwanth\Desktop\VAI ASSIGNMENTS> & C:\Users\Jashwanth\AppData\Local\Microsoft\WindowsApps\python3.12.exe "e:/User/Jashwanth/desktop/AI ASSIGNMENTS/Assignment-8.2.py"
Handled None input
Handled non-string input
PS E:\Users\Jashwanth\Desktop\VAI ASSIGNMENTS>
```

OBSERVATION:

Both to_uppercase() and to_lowercase() functions worked correctly for valid string inputs by converting text into upper and lower case.

Assert test cases verified the outputs automatically.

When invalid inputs like None and numbers were given, the functions raised TypeError.

Exception handling successfully caught these errors.

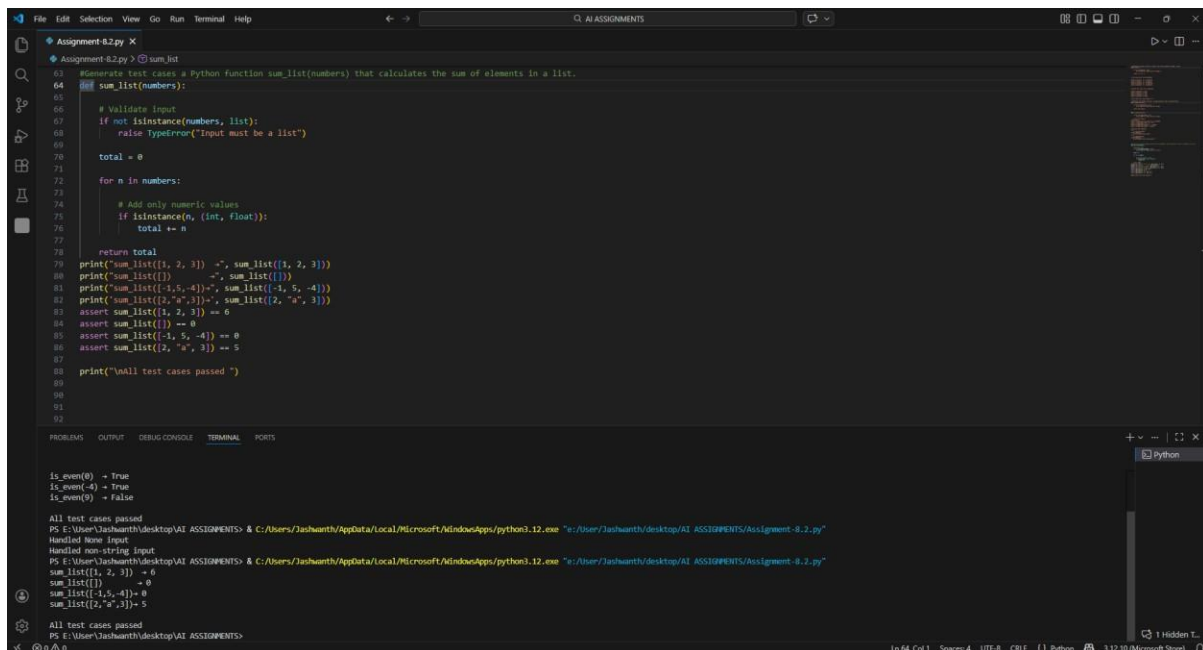
Overall, all test cases passed and the functions behaved as expected.

TASK-3

PROMPT:

Generate test cases a Python function sum_list(numbers) that calculates the sum of elements in a list.

CODE:



```
63 #Generate test cases a Python function sum_list(numbers) that calculates the sum of elements in a list.
64 def sum_list(numbers):
65     # Validates input
66     if not isinstance(numbers, list):
67         raise TypeError("Input must be a list")
68
69     total = 0
70
71     for n in numbers:
72         # Add only numeric values
73         if isinstance(n, (int, float)):
74             total += n
75
76     return total
77
78 print("sum_list([1, 2, 3])", "-", sum_list([1, 2, 3]))
79 print("sum_list([])", "-", sum_list([]))
80 print("sum_list([-1, 5, -4])", "-", sum_list([-1, 5, -4]))
81 print("sum_list([2, -2, 3])", "-", sum_list([2, -2, 3]))
82 assert sum_list([1, 2, 3]) == 6
83 assert sum_list([]) == 0
84 assert sum_list([-1, 5, -4]) == 0
85 assert sum_list([2, -2, 3]) == 3
86
87 print("All test cases passed")
88
89
90
91
92
```

```
is_even(0) -> True
is_even(-4) -> True
is_even(9) -> False

All test cases passed
PS E:\User\Jashwanth\desktop\AI ASSIGNMENTS> & C:\Users\Jashwanth\AppData\Local\Microsoft\WindowsApps\python3.12.exe "e:/user/jashwanth/desktop/AI ASSIGNMENTS/Assignment-8.2.py"
Handled non-string input
PS E:\User\Jashwanth\desktop\AI ASSIGNMENTS> & C:\Users\Jashwanth\AppData\Local\Microsoft\WindowsApps\python3.12.exe "e:/user/jashwanth/desktop/AI ASSIGNMENTS/Assignment-8.2.py"
sum_list([1, 2, 3]) -> 6
sum_list([]) -> 0
sum_list([-1, 5, -4]) -> 0
sum_list([2, -2, 3]) -> 3
All test cases passed
PS E:\User\Jashwanth\desktop\AI ASSIGNMENTS>
```

OBSERVATION:

The `sum_list(numbers)` function correctly calculated the sum of elements in a list.

It added only numeric values and ignored non-numeric data.

Test cases and assert statements verified the outputs.

Invalid input (non-list) raised a `TypeError`.

All test cases passed successfully, showing the function works as expected.

TASK-4

PROMPT:

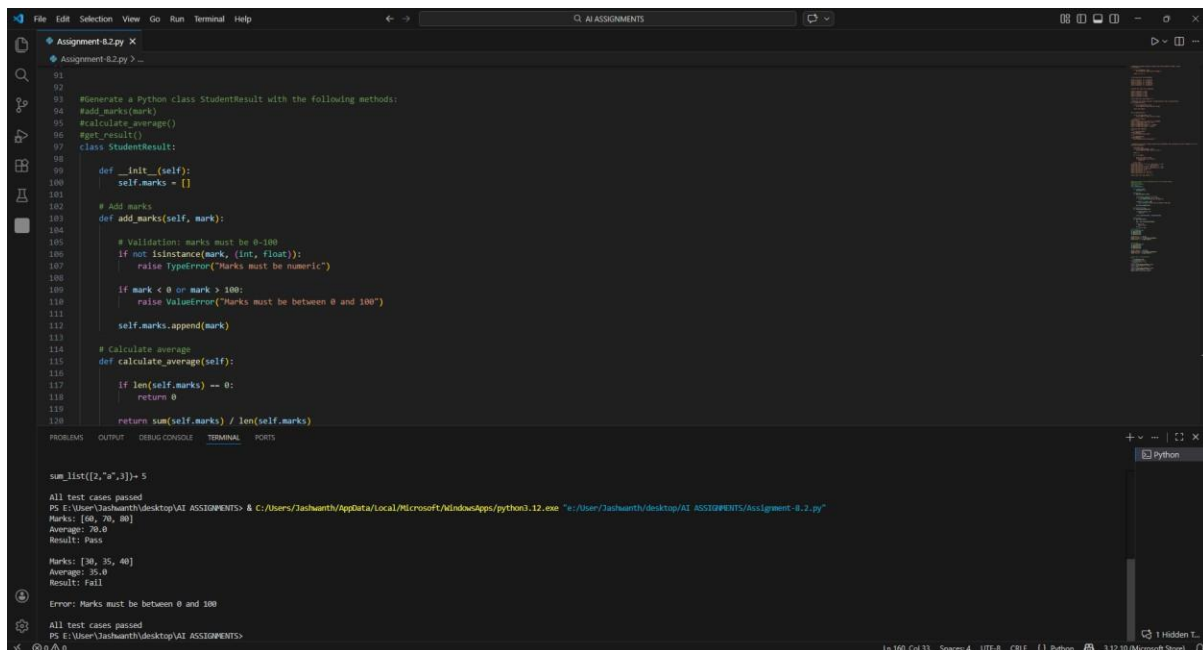
Generate a Python class `StudentResult` with the following methods:

`add_marks(mark)`

`calculate_average()`

`get_result()`

CODE:



```
91
92
93 #Generate a Python class StudentResult with the following methods:
94 #add_marks(mark)
95 #calculate_average()
96 #get_result()
97 class StudentResult:
98
99     def __init__(self):
100         self.marks = []
101
102     # Add marks
103     def add_marks(self, mark):
104
105         # Validation: marks must be 0-100
106         if not isinstance(mark, (int, float)):
107             raise TypeError("Marks must be numeric")
108
109         if mark < 0 or mark > 100:
110             raise ValueError("Marks must be between 0 and 100")
111
112         self.marks.append(mark)
113
114     # Calculate average
115     def calculate_average(self):
116
117         if len(self.marks) == 0:
118             return 0
119
120         return sum(self.marks) / len(self.marks)
121
122
123 sum_list([2,"a",3])= 5
124
125 All test cases passed
126 PS E:\User\Jashwanth\desktop\AI ASSIGNMENTS> & C:\Users\Jashwanth\AppData\Local\Microsoft\WindowsApps\python3.12.exe "e:\User\Jashwanth\desktop\AI ASSIGNMENTS\Assignment-8.2.py"
127 Marks: [80, 70, 80]
128 Average: 78.0
129 Result: Pass
130
131 Marks: [30, 35, 40]
132 Average: 35.0
133 Result: Fail
134
135 Error: Marks must be between 0 and 100
136
137 All test cases passed
138 PS E:\User\Jashwanth\desktop\AI ASSIGNMENTS>
```

OBSERVATION:

It contains a class named StudentResult with methods to add marks, calculate average, and get result. The code includes validation to ensure marks are numeric and between 0 and 100.

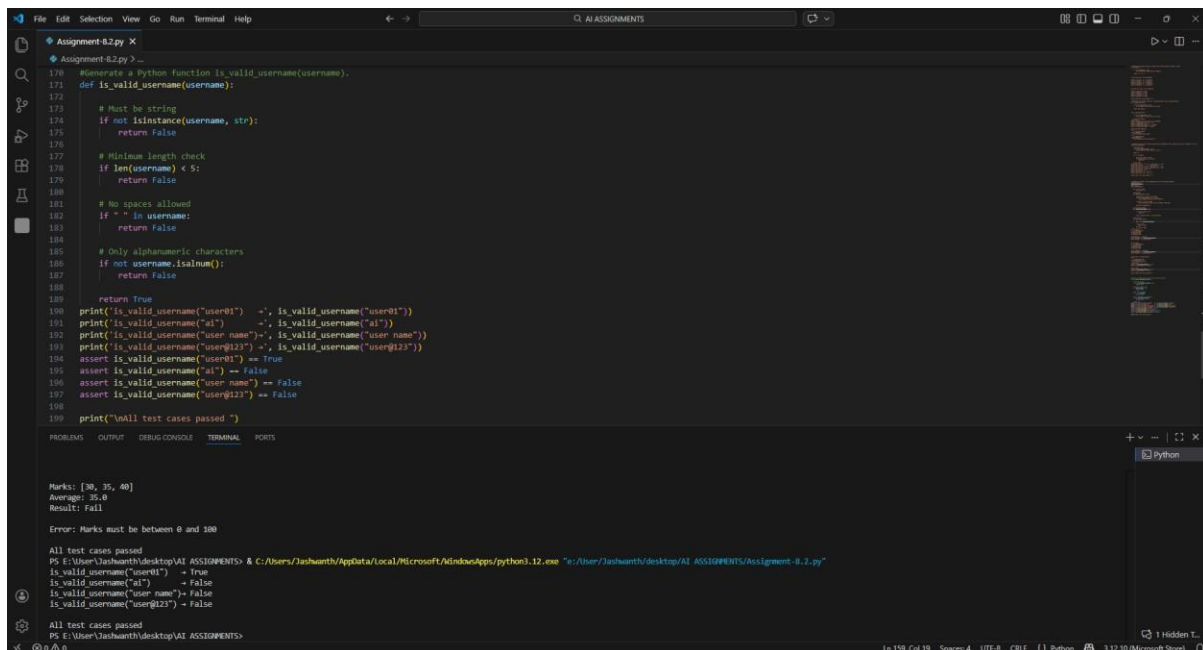
It shows marks being added, average calculated, and results displayed as Pass or Fail. An error message is also shown when invalid marks (outside 0-100) are entered. Overall, the tests appear to pass successfully.

TASK-5

PROMPT:

Generate a Python function is_valid_username(username).

CODE:



```
176 #Generate a Python function is_valid_username(username).
177 def is_valid_username(username):
178     # Must be string
179     if not isinstance(username, str):
180         return False
181     # Minimum length check
182     if len(username) < 5:
183         return False
184     # No spaces allowed
185     if " " in username:
186         return False
187     # Only alphanumeric characters
188     if not username.isalnum():
189         return False
190     return True
191 print(is_valid_username("user01"), is_valid_username("user01"))
192 print(is_valid_username("a1"), is_valid_username("a1"))
193 print(is_valid_username("user name"), is_valid_username("user name"))
194 print(is_valid_username("user@123"), is_valid_username("user@123"))
195 assert is_valid_username("user01") == True
196 assert is_valid_username("a1") == False
197 assert is_valid_username("user name") == False
198 assert is_valid_username("user@123") == False
199 print("All test cases passed")

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

Marks: [30, 35, 40]
Average: 35.0
Result: Fail

Error: Marks must be between 0 and 100

All test cases passed
PS E:\User\Jashwanth\desktop\AI ASSIGNMENTS> & C:\Users\Jashwanth\AppData\Local\Microsoft\WindowsApps\python3.12.exe "c:/user/jashwanth/desktop/AI ASSIGNMENTS/Assignment-8.2.py"
is_valid_username("user01") = True
is_valid_username("a1") = False
is_valid_username("user name") = False
is_valid_username("user@123") = False

All test cases passed
PS E:\User\Jashwanth\desktop\AI ASSIGNMENTS>
```

OBSERVATION:

is_valid_username(username).

The function checks if:

- The input is a string
- The length is at least 5 characters
- There are no spaces
- It contains only alphanumeric characters

The program defines a function to check whether a username is valid. It verifies string type, minimum length, no spaces, and only letters/numbers. Test cases are executed and all outputs are correct, showing the function works properly.